

Automatic exploration of reduced floating-point representations in iterative methods

Yohan Chatelain^{1,4} Eric Petit^{5,4} Pablo de Oliveira Castro^{1,4}
Ghislain Lartigue³ David Defour^{2,4}

¹Université de Versailles Saint-Quentin-en-Yvelines, Li-PaRAD

²Université de Perpignan Via Domitia, LAMPS

³Normandie Université, CORIA – CNRS

⁴Exascale Computing Research, ECR

⁵Intel Corporation

Euro-Par 2019

- ▶ Approximate computing use lower precision to reduce **time**, **communications' volume** and **energy** consumption
- ▶ Deep learning is driving the research in approximate computing
- ▶ Emergence of **new FP formats** that provide great speedup when combined with **custom hardware** (Google TPU [?], Intel's Nervanna Neural Network [?], IBM's SoC [?])

Objective: How can we harness these benefits for HPC?

Reducing precision is a three-dimensional problem:

- ▶ **Spatial**: Finding code parts that can be rewritten into lower precision [?, ?, ?, ?]
- ▶ **Temporal**: Finding time lapses where working precision can be lowered. Works [?, ?] based on *Iterative refinement* [?]
- ▶ **Precision**: Finding the precision to use. Most of works focus on existing IEEE-754 formats (mixed-precision) but new formats exist (bfloat16 [?], DLfloat [?]).

Needs: a fine-grain exploration at bitwise level.

VPREC

- ▶ Simulating **custom FP formats** with correct rounding
- ▶ Easily testing new formats on HPC codes

Methodology to find lower precisions

- ▶ **Automatic exploration** on iterative scheme
- ▶ Gives lowest precision at **bit level** for **each iteration**
- ▶ Validation of the **robustness** to round-off errors

Validation on an industrial code (YALES2)

- ▶ Scale mixed-precision version on CRIANN cluster
- ▶ **Communication's volume** reduction and **speedup**

Example: Newton-Raphson method

k	x_{k+1}	s_k^{10}	s_k^2
0	0.0690266447076745	0.11	0.37
1	0.1230846130203958	0.21	0.70
2	0.1985746566605835	0.43	1.43
3	0.2732703639721015	0.84	2.79
4	0.3119369815109966	1.79	5.95
5	0.3181822938100336	3.40	11.3
6	0.3183098350392471	6.79	22.6
7	0.3183098861837825	13.6	45.2
8	0.3183098861837907	15.6	51.8
9	0.3183098861837907	15.6	51.8

```
double newton(double x0) {
    double x_k;
    double x_k1=x0;
    double b=PI;
    do {
        x_k = x_k1;
        x_k1 = x_k*(2-b*x_k);
    } while (fabs((x_k1-x_k)/x_k)
              >= 1e-15);
    return x_k1;
}
```

Table: (left) Convergence speed of Newton-Raphson for the computation of the inverse of π using the IEEE-754 binary64 format (right). Highlighted digits in red are non significant

In the first iterations, most digits are **incorrect**, hinting that a low precision for the evaluation of f is enough

Question: How to find the minimal precision required to converge?

VPREC on the Newton-Raphson method

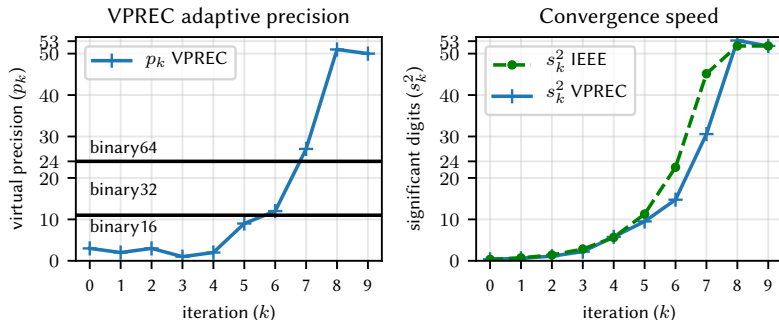
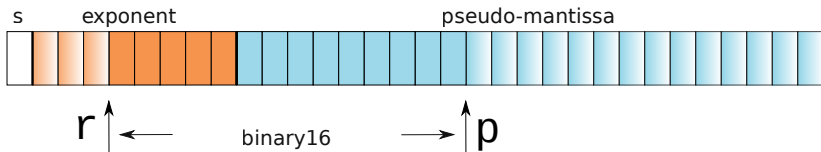


Figure: (left) Precision found with VPREC for Newton-Raphson. (right) Both the standard IEEE-754 binary64 version and the VPREC low precision configuration converge to 10^{-15} in nine iterations

Simulates any format that fits into a binary64

- ▶ Defined by couple (r, p) , r =exponent size, p =pseudo-mantissa size
- ▶ Correctly *round to nearest ties to even*
- ▶ Handles NaN, Inf and denormals



By setting $r = 5$ and $p = 10$, VPREC simulates a *binary16* embedded inside a *binary32*

VPREC rounding

Each FP operation $\bullet \in \{+, -, *, /\}$ is replaced by:

$$VPREC_{(r,p)}(x \bullet y) = \underbrace{\diamond_{(r,p)}}_{\text{OB}}(\underbrace{\diamond_{(r,p)}}_{\text{IB}}(x) \bullet \underbrace{\diamond_{(r,p)}}_{\text{IB}}(y))$$

where

$$\diamond_{(r,p)}(z) = \begin{cases} NaN_{b64} & \text{if } z = NaN_r \\ \pm\infty & \text{if } \lfloor \log |z| \rfloor > e_{max} \text{ or } z = \pm\infty \\ \pm 0 & \text{if } \lfloor \log |z| \rfloor < e_{min} + p \\ \circ_p(z) & \text{otherwise} \end{cases}$$

and

$$e_{max} = 2^{r-1} - 1, e_{min} = 1 - e_{max}$$

and

$\circ_p(x)$ is the *RoundToNearestTiesToEven* at precision p

Two modes : OutBound Precision (OB) or InBound Precision (IB)

VPREC rounding: example

Simulating binary16 inside a binary32

- ▶ $r = 5, p = 10$ ($e_{max} = 15, e_{min} = -14$)
- ▶ $VPREC_{5,10}((2.903225 + 2.903225) * 256) = ?$

- ▶ Verificarlo is a numerical toolbox for the floating point model:
 - ▶ **Numerical validation**: detect numerical bugs and ensure numerical accuracy during code modernization
 - ▶ **Numerical optimization**: reduce precision, harness new floating point formats and architectures
- ▶ Replaces every FP operations by a call to Verificarlo interface
- ▶ Interface allows using various backends during execution [?, ?]
- ▶ Operates on optimized code: evaluates the impact of compiler optimizations

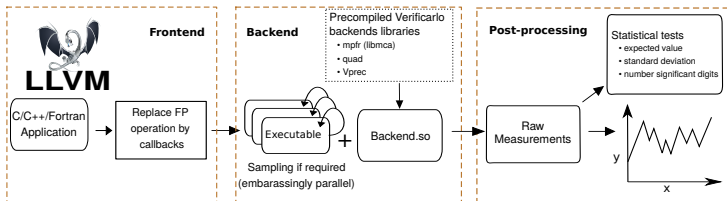


Figure: General Verificarlo exploration workflow with VPREC backend

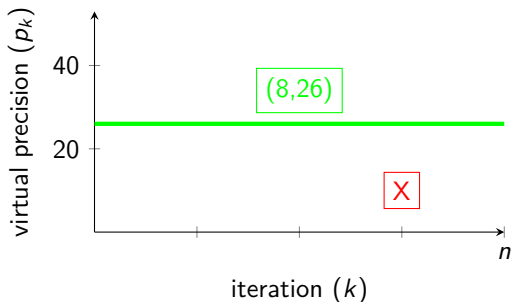
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



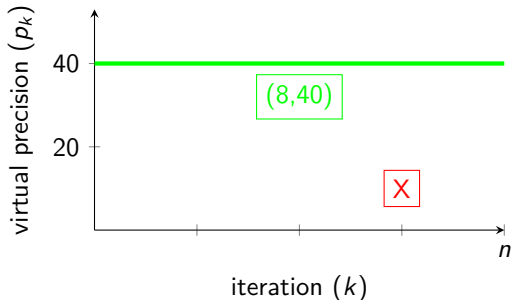
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



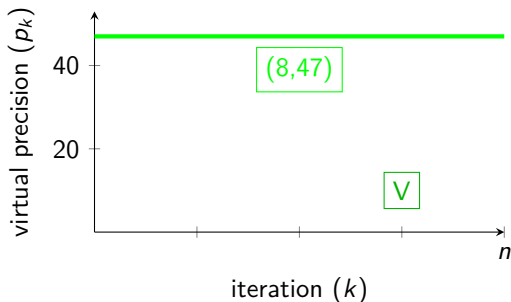
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



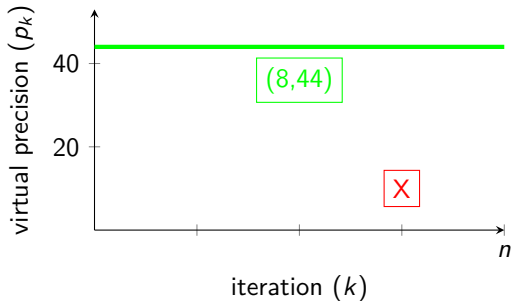
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



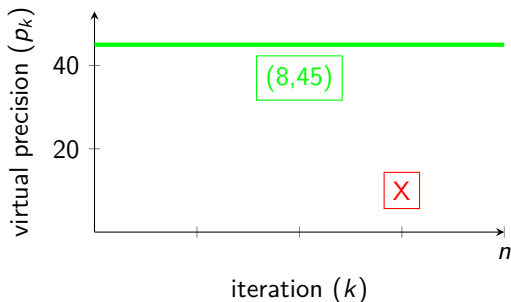
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



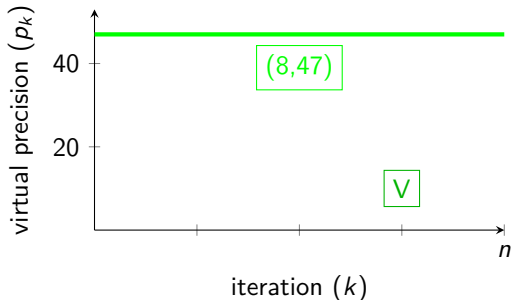
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



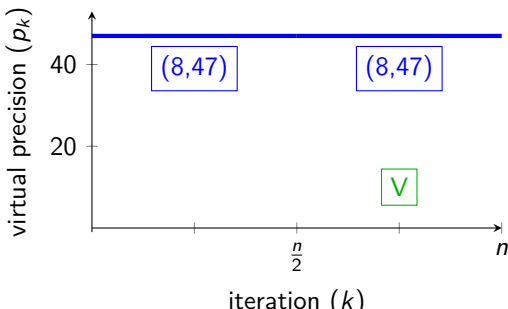
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



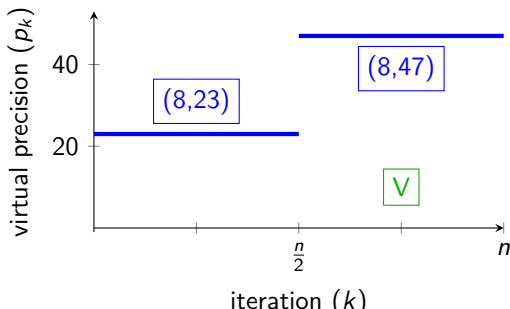
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



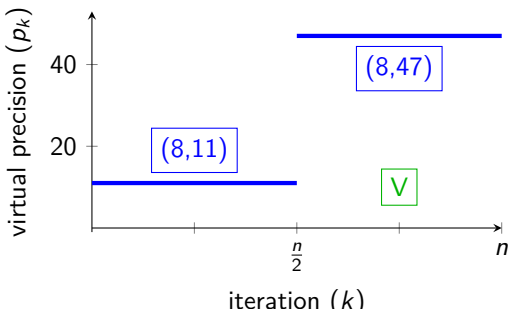
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



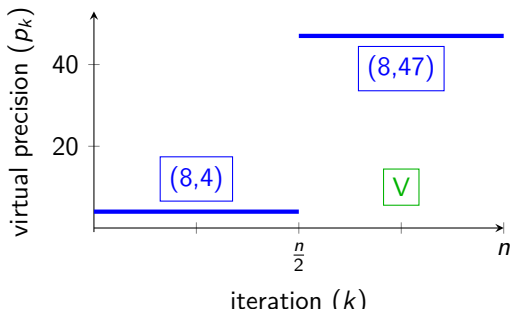
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



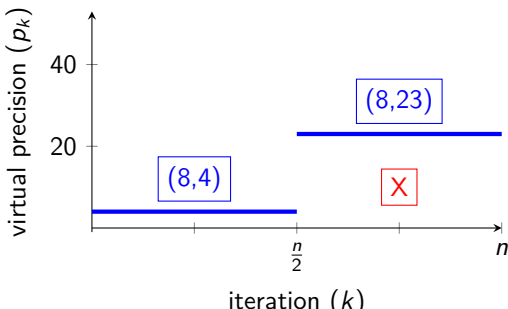
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



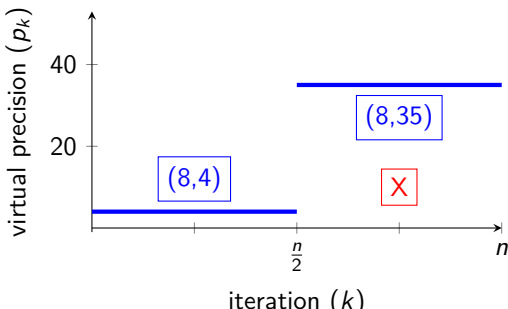
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**

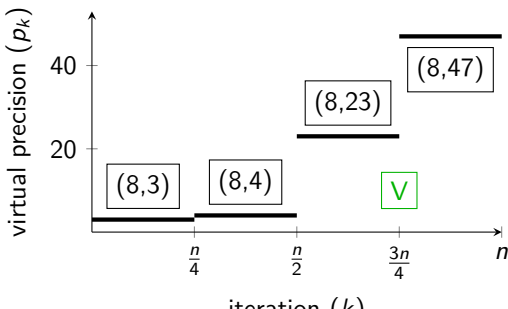
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



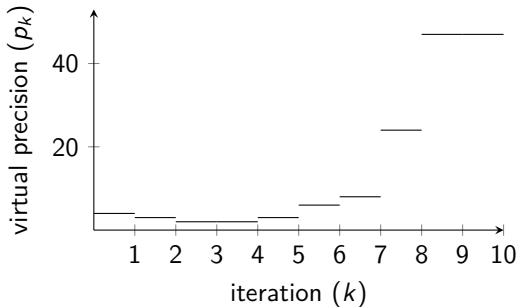
Lowest precision exploration

Objective: Find lowest valid configuration by enforcing four principles:

- ▶ Avoid combinatorial explosion ($(rp)^n$ combinations)
- ▶ Avoid configurations that oscillate quickly
- ▶ Distribute the reduction among all iterations
- ▶ Reduce more the first iterations than last iterations

Valid configuration

- ▶ Configuration $c = [(r_0, p_0), \dots, (r_n, p_n)]$
- ▶ Preserves **accuracy** and **convergence**



Exploration on an industrial use case: YALES2



Coria-CNRS and Safran, Solvay, GDF-Suez, www.coria-cfd.fr

Solver for two-phase combustion

- From primary atomization to pollutant prediction

Deflated Preconditioned Conjugate Gradient

- A fine grid is geometrically projected on a coarse grid
- A CG is applied on the coarse grid and on the fine grid

Preccinsta burner use case

- Evaluation on 1.75 million mesh elements
- Exploring reduction on *entire application* and *deflated grid*

Piecewise constant exploration on *preccinsta 1.75M*

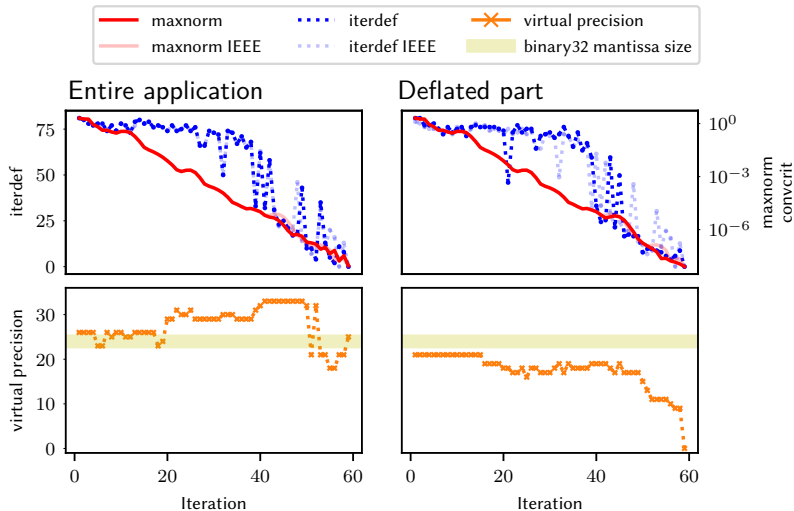


Figure: Adaptive precision searching on YALES2's DPCG on 1 MPI. $r = 8$. Reduced precision solution follows the reference IEEE convergence profile.

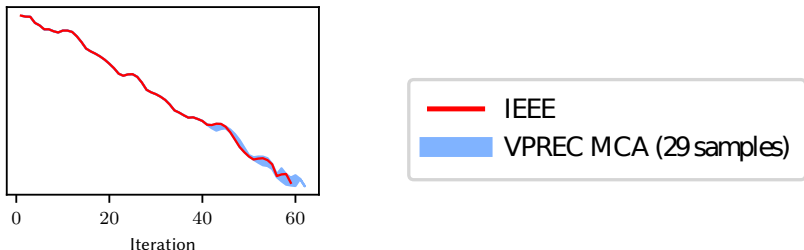
Validating resiliency to round-off errors

VPREC is a deterministic computation model

- ▶ How to validate configuration's robustness to rounding errors?

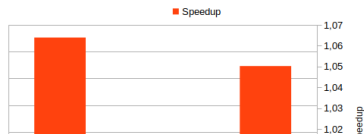
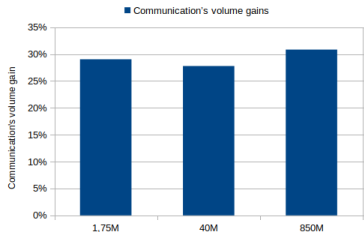
Using Monte Carlo Arithmetic [?]

- ▶ Introduces random noise to **model round-off errors** at each FP ops
- ▶ Used as **second step** for validating VPREC configurations
- ▶ 29 converged MCA samples gives 90% of probability with 0.95 confidence level based on Bernoulli trials [?] that configuration is robust to round-off errors



Performance validation on mixed-precision version

- ▶ **Mixed-version** to use both binary32 and binary64 formats in the deflated operator
- ▶ Switch from binary32 to binary64 when convergence criteria is below 10^{-7}
- ▶ Scaling from 1 to 560 cores on CRIANN cluster (66 bisocket Intel Xeon E5-2680 nodes and Intel Omnipath interconnect)



VPREC

- ▶ Simulates FP formats
- ▶ Easily tests custom formats on HPC codes

Find lowest precision exploration

- ▶ Automatic search to find lowest precision
- ▶ Reduces working precision while preserving accuracy and convergence
- ▶ Robustness to rounding errors thanks to MCA

Validation on industrial code Yales2

- ▶ Mixed-precision version scaling up to 560 cores
- ▶ Average communications' volume reduction by 30% and speedup of 1.05

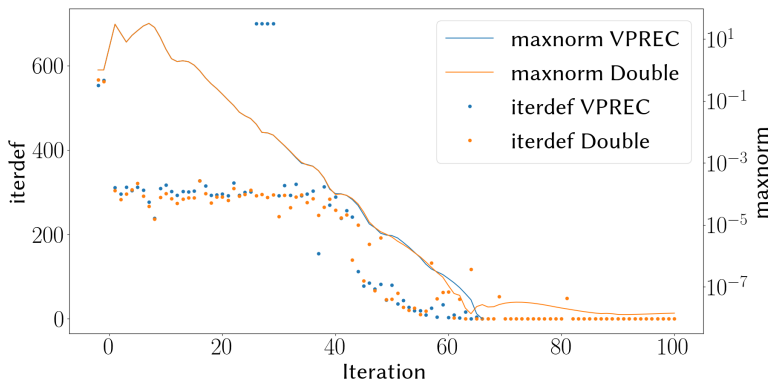
Backup slides

IEEE-754 formats

Name	Exponent (bits)	Pseudo-Mantissa (bits)	e_{min}, e_{max}
binary16	5	10	$(-14, +15)$
binary32	8	23	$(-126, +127)$
binary64	11	52	$(-1022, +1023)$
binary128	15	112	$(-16382, +16383)$

Table: Binary formats of the IEEE-754 norme

Accelerate the convergence



Exploration on *preccinsta* 850M on 112 MPI cores on deflated part. Solution found by the exploration (VPREC) converges in 66 iterations while the original (Double) converges in 100 iterations.

Reducing precision can be seen as a high-pass filter.

75% of solutions found by VPREC used less iterations than the IEEE version.

References I